



We're using spec-driven development (SDD) at Microsoft to preserve business intent, improve alignment, and help teams build software with greater speed, quality, and consistency.

Engineering the Frontier Firm: Sharing our AI-native approach to software development

September 3, 2026 | [Neil Orint](#) and [David Hirning](#)

About a year ago, our teams in Microsoft Digital—the company's IT organization—began an AI transformation the same way most organizations did: ad-hoc. Our engineers taught themselves new tools while those tools were still evolving.

It worked, up to a point. Individual developers got faster. But the gains stopped there.

"We quickly identified that improving the individual productivity of a developer was not resulting in a boost to team productivity. That was our hard lesson."

Sudhakar Sadasivuni, principal group engineering manager, Microsoft Digital



The problem wasn't the tools, but rather the process around them. The traditional software development lifecycle was built for human handoffs, and every handoff introduces a gap in intent.

"We quickly identified that improving the individual productivity of a developer was not resulting in a boost to team productivity," says Sudhakar Sadasivuni, a principal group engineering manager in Microsoft Digital, who leads our AI-native engineering initiative. "That was our hard lesson."

Eventually, our leadership posed the question: What would the lifecycle look like if it were designed from the start as a collaboration between humans and AI agents?

Exploring that concept led us to spec-driven development (SDD), an AI-native engineering approach that treats specifications as the primary artifact in the software development lifecycle. Instead of relying on code, documents, or prompts as the main source of truth, our teams now anchor development around a living specification, which captures the business intent and acceptance criteria before implementation begins.

Our engineers can now generate code faster, accelerate testing, and move from idea to implementation more quickly than ever before, thanks to the power and speed of AI.

This is a step towards Frontier Firm ways of working, where humans provide oversight and direction and AI tools handle task execution. And because we serve as the company's Customer Zero, the solution is a model that our customers can learn from and use in their own engineering organizations.

Shifting left

By shifting critical thinking and collaboration to the requirements phase of development, SDD resolves ambiguities, surfaces edge cases, and gets everyone on board early in the process, dramatically reducing late-stage chaos.

"Speed without direction is just expensive chaos—and we learned that the hard way through vibe coding. After burning our hands



on those traps, we realized that SDD gives you both velocity and vector.”

Mridul Verma, senior software engineer, Microsoft Digital

The result is a development process that helps teams align stakeholders earlier, reduce rework, and make more effective use of AI coding agents while maintaining a clear connection to business goals.

Another advantage of SDD, according to Mridul Verma, a senior software engineer in Microsoft Digital, is that it focuses not only on velocity, but also on vector. (Velocity is how fast your development moves, while vector reflects the direction you're moving.)

“Speed without direction is just expensive chaos—and we learned that the hard way through vibe coding,” Verma says. “After burning our hands on those traps, we realized that SDD gives you both velocity and vector.”

Why specs matter more in the AI era

Traditional software development often treats specifications as planning documents. After development begins, these documents can quickly become outdated as requirements evolve and implementation details change.

“In the AI era, the best dev teams aren't the ones that generate the most code. It's about how they're best able to preserve intent.”

Vignesh Vijayaraghavan, senior software engineer, Microsoft Digital



SDD flips that model.

The specification becomes a living artifact that evolves alongside the product, serving as a shared reference point for product managers, software engineers, architects, designers, testers, and AI coding agents.

That shift is especially important in AI-native development environments. While AI tools can generate large amounts of code quickly, they depend on clear instructions

and guardrails. Teams that can't consistently communicate intent typically discover that increased coding velocity simply produces more variability and additional rework.

"In the AI era, the best dev teams aren't the ones that generate the most code," says Vignesh Vijayaraghavan, a senior software engineer in Microsoft Digital. "It's about how they're best able to preserve intent."

There are five pillars that drive the transformational value of spec-driven development, based on our experience at Microsoft:

1

Spec as the source of truth

The spec becomes the single authoritative reference for all stakeholders.

2

Living, executable artifacts

Specs evolve alongside the project, remaining synchronized with the code and tests. They are co-authored by various stakeholders.

3

AI-assisted automation

SDD leverages AI coding agents to generate, test, and validate code, accelerating development and enhancing productivity.

4

Human validation and collaboration

Automation is central to SDD, human expertise is crucial for reviewing specs, refining requirements, and validating outcomes.

5

Predictability and measurability

SDD enables teams to track progress, measure outcomes, and improve predictability.

How the SDD workflow works

At Microsoft Digital, our developers and PMs begin by creating specifications that capture business goals, user requirements, edge cases, and acceptance tests. These specifications are version-controlled and continuously updated as the project evolves. Rather than serving as static documentation, they guide every major decision throughout the lifecycle.

"The spec becomes the shared source of truth for the entire team. Instead of handing requirements from one group to another, everyone collaborates around a living document that guides development and keeps decisions aligned."

Garima Gaurav, senior product manager, Microsoft Digital



It's important to note that before creating the specs, the team agrees on a constitution: A shared set of architectural principles, governance requirements, security standards, and development limitations that everyone adheres to. Developing this constitution is a team-wide exercise rather than an individual one, allowing employees and AI agents to work from the same facts and the same context.

"The spec becomes the shared source of truth for the entire team," says Garima Gaurav, a senior product manager in Microsoft Digital. "Instead of handing requirements from one group to another, everyone collaborates around a living document that guides development and keeps decisions aligned."

Our teams use GitHub Spec Kit to support the process. This workflow includes six major stages:

1. Define the business problem and expected outcomes
2. Clarify ambiguities and unanswered questions

3. Create a technical implementation plan
4. Break work into traceable tasks
5. Validate alignment between requirements and plans
6. Implement and test the solution

“As teams mature, they learn the right level of granularity, and breaking large requirements into multiple specs becomes a key practice for maintaining quality and preserving intent.”

Ajeya Kumar, principal software engineer, Microsoft Digital



Because each stage builds on the spec, teams can trace implementation decisions back to their related business requirements more easily than with traditional code-first approaches. AI coding agents enhance the spec and then generate code, tests, documentation, and other tasks from it. (Specs are version-controlled in the same repository to aid tracking and avoid confusion.) Engineers focus on validating intent, reviewing outputs, and refining requirements.

“Effective SDD adoption relies on well-scoped specifications,” says Ajeya Kumar, a principal software engineer in Microsoft Digital. “Smaller, focused specs generate outputs that are easier to review and validate, enabling teams to build incrementally rather than all at once. As teams mature, they learn the right level of granularity, and breaking large requirements into multiple specs becomes a key practice for maintaining quality and preserving intent.”

How SDD impacts different roles

Adopting spec-driven development across our organization involved more than just introducing new tools. Existing roles were tightly coupled to the old software development lifecycle, so some had to change while other new roles and skills were created. Microsoft Digital teams documented these changes at every step, because the shift in mindset is what makes SDD work.

Overall, we found that the biggest change was behavioral: Specifications stopped being documents that simply supported development and actually became the artifact

that drove it.

Here's how Microsoft's shift toward an SDD-driven approach has impacted various roles across the company:

Leadership

For engineering leaders, success is no longer measured solely by development activity. Leaders need to create an environment that rewards clarity, alignment, and collaboration before actual implementation begins. Dev teams are encouraged to invest time in resolving ambiguities and documenting expectations early in the software development lifecycle because this reduces costly rework later.

Leaders also play an important role in driving adoption. Our dev teams say that SDD delivered the greatest value when it became a standard way of working rather than an optional process that individuals can bypass. The adoption has to be deliberate.

"It's not something where you decide, today I woke up and I'll start using SDD," says Apoorv Gupta, a principal software engineer in Microsoft Digital. "It doesn't work like that. It's a mindset shift and a learning curve."

Developers

For developers, the transition to SDD often requires breaking long-standing work patterns. For years, software engineers were encouraged to move quickly from requirements to implementation. Under SDD, their first step is developing a complete understanding of the problem, including defining success, before any code is created.

"As developers, we get excited and want to jump directly into coding," Verma says. "Changing that habit was really tricky."

That change runs deeper than just a habit. The developers must make sure the agent fully understands the task before starting the coding process.

"SDD makes you clarify and refine your thoughts first for agents, not for yourself," Sadasivuni says. "This is agent-first solutioning, not human-first solutioning."

Our Microsoft Digital engineers now spend more time refining requirements, identifying edge cases, evaluating plans, and validating AI-generated outputs. Their

role has shifted away from producing every line of code to making sure that code implementation accurately reflects intent.

Program and product managers

Our program and product managers continue to perform many of their traditional responsibilities, including stakeholder management, prioritization, and roadmap planning. What's changed is the degree of ownership that they maintain throughout the development lifecycle.

"In spec-driven development, PMs become the owners of the spec; they're no longer just responsible for planning and prioritization. Everything starts with the spec, so they become the starting point for the entire system."

Apoorv Gupta, principal software engineer, Microsoft Digital



In an SDD environment, product managers become stewards of the specification itself. They help define success criteria, resolve ambiguities, document business requirements, and check to make sure that priorities remain visible throughout implementation. The specification becomes a shared contract among stakeholders and a common source of truth for the entire team.

"In spec-driven development, PMs become the owners of the spec; they're no longer just responsible for planning and prioritization," Gupta says. "Everything starts with the spec, so they become the starting point for the entire system. If the specs are right, then everything falls into place."

Architects

Architects establish the guardrails that govern the development process. One of the SDD first activities is defining a constitution that captures architectural principles, governance requirements, security standards, and development constraints. Architects are responsible for creating and maintaining those guardrails.

Because architectural decisions are made earlier in the process, teams can identify issues before implementation begins rather than discovering them during code

reviews or testing cycles.

Looking ahead

Spec-driven development continues to evolve at Microsoft. But our development teams already view it as a vital component of AI-native engineering. The goal is to create a system that preserves intent as work moves from business requirements to implementation.

"SDD gives PMs a level of empowerment that wasn't there before. You can move from an idea to a prototype, show it to customers and leaders, and start generating feedback much faster. Once teams experience that, they keep creating value, generating new ideas, and building on what they've learned."

Gopal Panigrahy, principal product manager, Microsoft Digital



As AI coding agents become more capable, clear specifications become more valuable. Teams that can define requirements, establish guardrails, and maintain shared ownership of intent will be better positioned to scale AI-assisted engineering without sacrificing quality or governance.

For Microsoft Digital, SDD represents an important step toward Frontier Firm ways of working, where humans provide direction, judgment, and accountability while AI helps accelerate execution. Preserving intent, not simply generating code, is what enables teams to move faster while continuing to deliver outcomes that meet business needs.

"SDD gives PMs a level of empowerment that wasn't there before," says Gopal Panigrahy, a principal product manager in Microsoft Digital. "You can move from an idea to a prototype, show it to customers and leaders, and start generating feedback much faster. Once teams experience that, they keep creating value, generating new ideas, and building on what they've learned."

Key takeaways

Here are some important guidelines as you consider spec-driven development for your own organization:

- **Make the specification the source of truth.** Treat the spec as a living artifact that evolves with the project rather than static documentation.
- **Resolve ambiguity before implementation begins.** Invest time up front identifying requirements, constraints, edge cases, and success criteria.
- **Assign clear ownership.** Product managers, architects, developers, and designers all play important roles in maintaining the specification.
- **Establish architectural guardrails early.** Define governance, security, and design principles before generating code.
- **Use AI to accelerate execution, not replace judgment.** Human expertise remains essential for validating outputs and ensuring alignment with business intent.

Try it out

[Explore GitHub Copilot and learn how spec-driven development can help your teams build, test, and document software more efficiently.](#)

Related links

- [Learn how we're powering the new age of AI-led engineering in IT at Microsoft.](#)
- [Check out our Dev Blog post about spec-driven development.](#)
- [Read our playbook to becoming a Frontier Firm in the era of AI.](#)
- [Find out how we're becoming an AI-first organization at Microsoft.](#)

Tags: [AI](#), [Automation](#), [Customer Zero](#), [Developer and automation](#), [Developer tools](#), [Developers and engineers](#)

Related Stories

The agentic future: How we're becoming an AI-first Frontier Firm at Microsoft

Powering the new age of AI-led engineering in IT at Microsoft

Inside Microsoft: Being Customer Zero in an AI-powered world

Accelerating transformation: How we're reshaping Microsoft with continuous improvement and AI



Engage with our experts!

Customers or Microsoft account team representatives from Fortune 500 companies are welcome to [request a virtual engagement](#) on this topic with experts from our Microsoft Digital team.



We'd like to hear from you!

[Want more information? Email us and include a link to this story and we'll get back to you.](#)

What's new

Surface Pro
Surface Laptop
Surface Laptop Ultra
Surface RTX Spark Dev Box
Copilot for organizations
Copilot for personal use
Explore Microsoft products
Windows 11 apps

Microsoft Store

Account profile
Download Center
Microsoft Store support
Returns
Order tracking
Certified Refurbished
Microsoft Store Promise
Flexible Payments

Education

Microsoft in education
Devices for education
Microsoft Teams for Education
Microsoft 365 Education
How to buy for your school
Educator training and development
Deals for students and parents
AI for education

Business

Microsoft AI

Developer & IT

Azure

Company

Careers

Microsoft Security	Microsoft Developer	About Microsoft
Dynamics 365	Microsoft Learn	Company news
Microsoft 365	Support for AI marketplace apps	Privacy at Microsoft
Microsoft Power Platform	Microsoft Tech Community	Investors
Microsoft Teams	Microsoft Marketplace	Diversity and inclusion
Microsoft 365 Copilot	Software companies	Accessibility
Small Business	Visual Studio	Sustainability

 [English \(United States\)](#)

 [Your Privacy Choices](#)

[Consumer Health Privacy](#)